

10. Testability

Testing leads to failure, and failure leads to understanding

—Burt Rutan

Industry estimates indicate that between 30 and 50 percent (or in some cases, even more) of the cost of developing well-engineered systems is taken up by testing. If the software architect can reduce this cost, the payoff is large.

Software testability refers to the ease with which software can be made to demonstrate its faults through (typically execution-based) testing. Specifically, testability refers to the probability, assuming that the software has at least one fault, that it will fail on its next test execution. Intuitively, a system is testable if it “gives up” its faults easily. If a fault is present in a system, then we want it to fail during testing as quickly as possible. Of course, calculating this probability is not easy and, as you will see when we discuss response measures for testability, other measures will be used.

Figure 10.1 shows a model of testing in which a program processes input and produces output. An oracle is an agent (human or mechanical) that decides whether the output is correct or not by comparing the output to the program’s specification. Output is not just the functionally produced value, but it also can include derived measures of quality attributes such as how long it took to produce the output. Figure 10.1 also shows that the program’s internal state can also be shown to the oracle, and an oracle can decide whether that is correct or not—that is, it can detect whether the program has entered an erroneous state and render a judgment as to the correctness of the program.

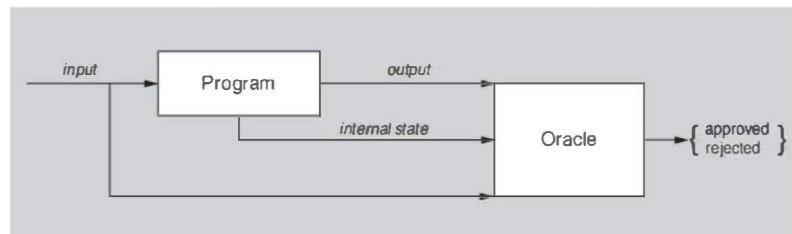


Figure 10.1. A model of testing

Setting and examining a program’s internal state is an aspect of testing that will figure prominently in our tactics for testability.

For a system to be properly testable, it must be possible to control each component’s inputs (and possibly manipulate its internal state) and then to observe its outputs (and possibly its internal state, either after or on the way to computing the outputs). Frequently this control and observation is done through the use of a test harness, which is specialized software (or in some cases, hardware) designed to exercise the software under test. Test harnesses come in various forms, such as a record-and-playback capability for data sent across various interfaces, or a simulator for an external environment in which a

piece of embedded software is tested, or even during production (see sidebar). The test harness can provide assistance in executing the test procedures and recording the output. A test harness can be a substantial piece of software in its own right, with its own architecture, stakeholders, and quality attribute requirements.

Testing is carried out by various developers, users, or quality assurance personnel. Portions of the system or the entire system may be tested. The response measures for testability deal with how effective the tests are in discovering faults and how long it takes to perform the tests to some desired level of coverage. Test cases can be written by the developers, the testing group, or the customer. The test cases can be a portion of acceptance testing or can drive the development as they do in certain types of Agile methodologies.

Netflix's Simian Army

Netflix distributes movies and television shows both via DVD and via streaming video. Their streaming video service has been extremely successful. In May 2011 Netflix streaming video accounted for 24 percent of the Internet traffic in North America. Naturally, high availability is important to Netflix.

Netflix hosts their computer services in the Amazon EC2 cloud, and they utilize what they call a "Simian Army" as a portion of their testing process. They began with a Chaos Monkey, which randomly kills processes in the running system. This allows the monitoring of the effect of failed processes and gives the ability to ensure that the system does not fail or suffer serious degradation as a result of a process failure.

Recently, the Chaos Monkey got some friends to assist in the testing. Currently, the Netflix Simian Army includes these:

- The Latency Monkey induces artificial delays in the client-server communication layer to simulate service degradation and measures if upstream services respond appropriately.
- The Conformity Monkey finds instances that don't adhere to best practices and shuts them down. For example, if an instance does not belong to an auto-scaling group, it will not appropriately scale when demand goes up.
- The Doctor Monkey taps into health checks that run on each instance as well as monitors other external signs of health (e.g., CPU load) to detect unhealthy instances.
- The Janitor Monkey ensures that the Netflix cloud environment is running free of clutter and waste. It searches for unused resources and disposes of them.
- The Security Monkey is an extension of Conformity Monkey. It finds security violations or vulnerabilities, such as improperly configured security groups, and terminates the offending instances. It also ensures that all the SSL and digital rights management (DRM) certificates are valid and are not coming up for renewal.
- The 10-18 Monkey (localization-internationalization) detects configuration and runtime problems in instances serving customers in multiple geographic regions, using different languages and character sets. The name 10-18 comes from *L10n-i18n*,

a sort of shorthand for the words *localization* and *internationalization*.

Some of the members of the Simian Army use fault injection to place faults into the running system in a controlled and monitored fashion. Other members monitor various specialized aspects of the system and its environment. Both of these techniques have broader applicability than just Netflix.

Not all faults are equal in terms of severity. More emphasis should be placed on finding the most severe faults than on finding other faults. The Simian Army reflects a determination by Netflix that the faults they look for are the most serious in terms of their impact.

This strategy illustrates that some systems are too complex and adaptive to be tested fully, because some of their behaviors are emergent. An aspect of testing in that arena is logging of operational data produced by the system, so that when failures occur, the logged data can be analyzed in the lab to try to reproduce the faults. Architecturally this can require mechanisms to access and log certain system state. The Simian Army is one way to discover and log behavior in systems of this ilk.

—LB

Testing of code is a special case of validation, which is making sure that an engineered artifact meets the needs of its stakeholders or is suitable for use. In [Chapter 21](#) we will discuss architectural design reviews. This is another kind of validation, where the artifact being tested is the architecture. In this chapter we are concerned only with the testability of a running system and of its source code.

10.1. Testability General Scenario

We can now describe the general scenario for testability.

- *Source of stimulus.* The testing is performed by unit testers, integration testers, or system testers (on the developing organization side), or acceptance testers and end users (on the customer side). The source could be human or an automated tester.
- *Stimulus.* A set of tests is executed due to the completion of a coding increment such as a class layer or service, the completed integration of a subsystem, the complete implementation of the whole system, or the delivery of the system to the customer.
- *Artifact.* A unit of code (corresponding to a module in the architecture), a subsystem, or the whole system is the artifact being tested.
- *Environment.* The test can happen at development time, at compile time, at deployment time, or while the system is running (perhaps in routine use). The environment can also include the test harness or test environments in use.
- *Response.* The system can be controlled to perform the desired tests and the results from the test can be observed.
- *Response measure.* Response measures are aimed at representing how easily a system under test “gives up” its faults. Measures might include the effort involved in finding a fault or a particular class of faults, the effort required to test a given percentage of statements, the length of the longest test chain (a measure of the difficulty of performing the tests), measures of effort to perform the tests, measures of effort to actually find faults, estimates of the probability of finding additional faults, and the length of time or amount of effort to prepare the test environment.

Maybe one measure is the ease at which the system can be brought into a specific state. In addition, measures of the reduction in risk of the remaining errors in the system can be used. Not all faults are equal in terms of their possible impact. Measures of risk reduction attempt to rate the severity of faults found (or to be found).

Figure 10.2 shows a concrete scenario for testability. The unit tester completes a code unit during development and performs a test sequence whose results are captured and that gives 85 percent path coverage within three hours of testing.

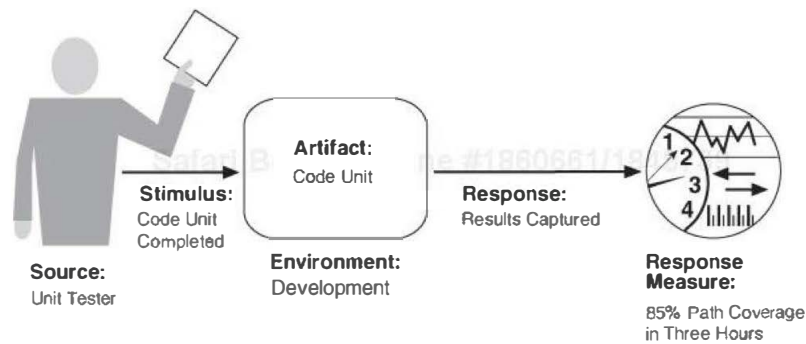


Figure 10.2. Sample concrete testability scenario

Table 10.1 enumerates the elements of the general scenario that characterize testability.

Table 10.1. Testability General Scenario

Portion of Scenario	Possible Values
Source	Unit testers, integration testers, system testers, acceptance testers, end users, either running tests manually or using automated testing tools
Stimulus	A set of tests is executed due to the completion of a coding increment such as a class layer or service, the completed integration of a subsystem, the complete implementation of the whole system, or the delivery of the system to the customer.
Environment	Design time, development time, compile time, integration time, deployment time, run time
Artifacts	The portion of the system being tested
Response	One or more of the following: execute test suite and capture results, capture activity that resulted in the fault, control and monitor the state of the system
Response Measure	One or more of the following: effort to find a fault or class of faults, effort to achieve a given percentage of state space coverage, probability of fault being revealed by the next test, time to perform tests, effort to detect faults, length of longest dependency chain in test, length of time to prepare test environment, reduction in risk exposure ($\text{size}(\text{loss}) \times \text{prob}(\text{loss})$)

10.2. Tactics for Testability

The goal of tactics for testability is to allow for easier testing when an increment of software development is completed. Figure 10.3 displays the use of tactics for testability. Architectural techniques for enhancing the software testability have not received as much attention as more mature quality attribute disciplines such as modifiability, performance, and availability, but as we stated before, anything the architect can do to reduce the high cost of testing will yield a significant benefit.

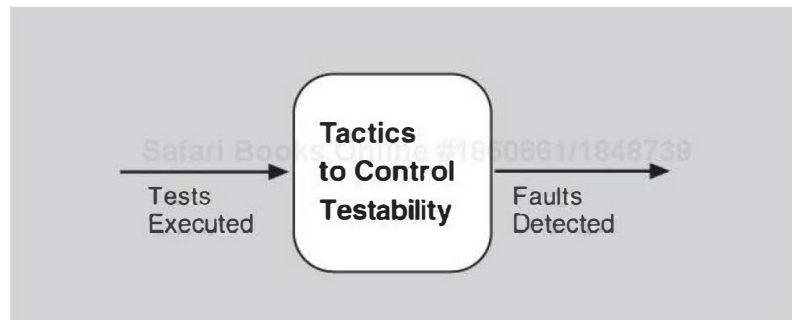


Figure 10.3. The goal of testability tactics

There are two categories of tactics for testability. The first category deals with adding controllability and observability to the system. The second deals with limiting complexity in the system's design.

Control and Observe System State

Control and observation are so central to testability that some authors even define testability in those terms. The two go hand-in-hand; it makes no sense to control something if you can't observe what happens when you do. The simplest form of control and observation is to provide a software component with a set of inputs, let it do its work, and then observe its outputs. However, the control and observe system state category of testability tactics provides insight into software that goes beyond its inputs and outputs. These tactics cause a component to maintain some sort of state information, allow testers to assign a value to that state information, and/or make that information accessible to testers on demand. The state information might be an operating state, the value of some key variable, performance load, intermediate process steps, or anything else useful to re-creating component behavior. Specific tactics include the following:

- *Specialized interfaces.* Having specialized testing interfaces allows you to control or capture variable values for a component either through a test harness or through normal execution. Examples of specialized test routines include these:
 - A *set* and *get* method for important variables, modes, or attributes (methods that might otherwise not be available except for testing purposes)
 - A *report* method that returns the full state of the object
 - A *reset* method to set the internal state (for example, all the attributes of a class) to a specified internal state

- A method to turn on verbose output, various levels of event logging, performance instrumentation, or resource monitoring

Specialized testing interfaces and methods should be clearly identified or kept separate from the access methods and interfaces for required functionality, so that they can be removed if needed. (However, in performance-critical and some safety-critical systems, it is problematic to field different code than that which was tested. If you remove the test code, how will you know the code you field has the same behavior, particularly the same timing behavior, as the code you tested? For other kinds of systems, however, this strategy is effective.)

- *Record/playback*. The state that caused a fault is often difficult to re-create. Recording the state when it crosses an interface allows that state to be used to “play the system back” and to re-create the fault. Record/playback refers to both capturing information crossing an interface and using it as input for further testing.
- *Localize state storage*. To start a system, subsystem, or module in an arbitrary state for a test, it is most convenient if that state is stored in a single place. By contrast, if the state is buried or distributed, this becomes difficult if not impossible. The state can be fine-grained, even bit-level, or coarse-grained to represent broad abstractions or overall operational modes. The choice of granularity depends on how the states will be used in testing. A convenient way to “externalize” state storage (that is, to make it able to be manipulated through interface features) is to use a state machine (or state machine object) as the mechanism to track and report current state.
- *Abstract data sources*. Similar to controlling a program’s state, easily controlling its input data makes it easier to test. Abstracting the interfaces lets you substitute test data more easily. For example, if you have a database of customer transactions, you could design your architecture so that it is easy to point your test system at other test databases, or possibly even to files of test data instead, without having to change your functional code.
- *Sandbox*. “Sandboxing” refers to isolating an instance of the system from the real world to enable experimentation that is unconstrained by the worry about having to undo the consequences of the experiment. Testing is helped by the ability to operate the system in such a way that it has no permanent consequences, or so that any consequences can be rolled back. This can be used for scenario analysis, training, and simulation. (The Spring framework, which is quite popular in the Java community, comes with a set of test utilities that support this. Tests are run as a “transaction,” which is rolled back at the end.)

A common form of sandboxing is to virtualize resources. Testing a system often involves interacting with resources whose behavior is outside the control of the system. Using a sandbox, you can build a version of the resource whose behavior is under your control. For example, the system clock’s behavior is typically not under our control—it increments one second each second—which means that if we want to make the system think it’s midnight on the day when all of the data structures are supposed to overflow, we need a way to do that, because waiting around is a poor choice. By having the capability to abstract system time from clock time, we can allow the system (or components) to run at faster than wall-clock time, and to allow the system (or components) to be tested at critical time boundaries (such as the next

shift on or off Daylight Savings Time). Similar virtualizations could be done for other resources, such as memory, battery, network, and so on. Stubs, mocks, and dependency injection are simple but effective forms of virtualization.

- *Executable assertions.* Using this tactic, assertions are (usually) hand-coded and placed at desired locations to indicate when and where a program is in a faulty state. The assertions are often designed to check that data values satisfy specified constraints. Assertions are defined in terms of specific data declarations, and they must be placed where the data values are referenced or modified. Assertions can be expressed as pre- and post-conditions for each method and also as class-level invariants. This results in increasing observability, when an assertion is flagged as having failed. Assertions systematically inserted where data values change can be seen as a manual way to produce an “extended” type. Essentially, the user is annotating a type with additional checking code. Any time an object of that type is modified, the checking code is automatically executed, and warnings are generated if any conditions are violated. To the extent that the assertions cover the test cases, they effectively embed the test oracle in the code—assuming the assertions are correct and correctly coded.

All of these tactics add capability or abstraction to the software that (were we not interested in testing) otherwise would not be there. They can be seen as replacing bare-bones, get-the-job-done software with more elaborate software that has bells and whistles for testing. There are a number of techniques for effecting this replacement. These are not testability tactics, per se, but techniques for replacing one component with a different version of itself. They include the following:

- Component replacement, which simply swaps the implementation of a component with a different implementation that (in the case of testability) has features that facilitate testing. Component replacement is often accomplished in a system’s build scripts.
- Preprocessor macros that, when activated, expand to state-reporting code or activate probe statements that return or display information, or return control to a testing console.
- Aspects (in aspect-oriented programs) that handle the cross-cutting concern of how state is reported.

Limit Complexity

Complex software is harder to test. This is because, by the definition of complexity, its operating state space is very large and (all else being equal) it is more difficult to re-create an exact state in a large state space than to do so in a small state space. Because testing is not just about making the software fail but about finding the fault that caused the failure so that it can be removed, we are often concerned with making behavior repeatable. This category has three tactics:

- *Limit structural complexity.* This tactic includes avoiding or resolving cyclic dependencies between components, isolating and encapsulating dependencies on the external environment, and reducing dependencies between components in general (for example, reduce the number of external accesses to a module’s public data). In object-oriented systems, you can simplify the inheritance hierarchy: Limit the number of classes from which a class is derived, or the number of classes derived from a class. Limit the depth of the inheritance tree, and the number of children of a class. Limit polymorphism and dynamic calls. One structural metric that has been

shown empirically to correlate to testability is called the *response* of a class. The response of class C is a count of the number of methods of C plus the number of methods of other classes that are invoked by the methods of C. Keeping this metric low can increase testability.

Having high cohesion, loose coupling, and separation of concerns—all modifiability tactics (see [Chapter 7](#))—can also help with testability. They are a form of limiting the complexity of the architectural elements by giving each element a focused task with limited interaction with other elements. Separation of concerns can help achieve controllability and observability (as well as reducing the size of the overall program’s state space). Controllability is critical to making testing tractable, as Robert Binder has noted: “A component that can act independently of others is more readily controllable. . . . With high coupling among classes it is typically more difficult to control the class under test, thus reducing testability. . . . If user interface capabilities are entwined with basic functions it will be more difficult to test each function” [Binder 94].

Also, systems that require complete data consistency at all times are often more complex than those that do not. If your requirements allow it, consider building your system under the “eventual consistency” model, where sooner or later (but maybe not right now) your data will reach a consistent state. This often makes system design simpler, and therefore easier to test.

Finally, some architectural styles lend themselves to testability. In a layered style, you can test lower layers first, then test higher layers with confidence in the lower layers.

- *Limit nondeterminism.* The counterpart to limiting structural complexity is limiting behavioral complexity, and when it comes to testing, nondeterminism is a very pernicious form of complex behavior. Nondeterministic systems are harder to test than deterministic systems. This tactic involves finding all the sources of nondeterminism, such as unconstrained parallelism, and weeding them out as much as possible. Some sources of nondeterminism are unavoidable—for instance, in multi-threaded systems that respond to unpredictable events—but for such systems, other tactics (such as record/playback) are available.

[Figure 10.4](#) provides a summary of the tactics used for testability.

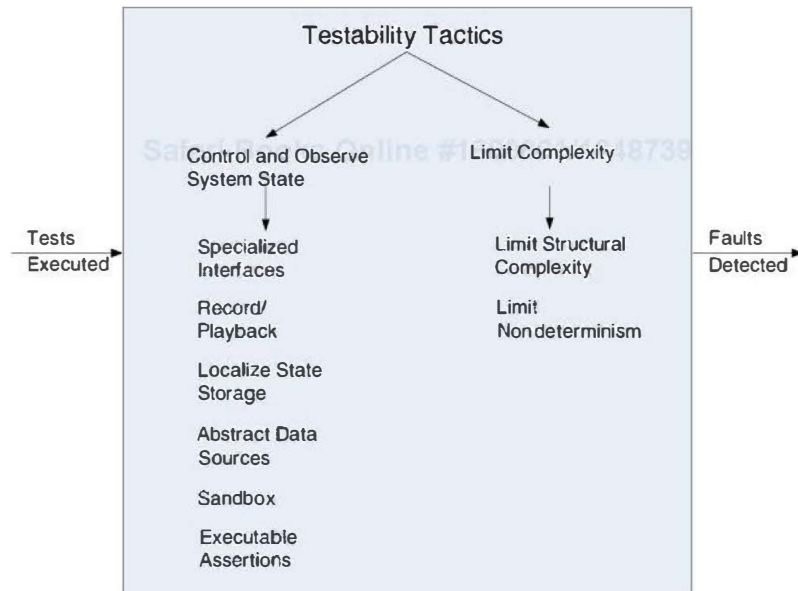


Figure 10.4. Testability tactics

10.3. A Design Checklist for Testability

Table 10.2 is a checklist to support the design and analysis process for testability.

Table 10.2. Checklist to Support the Design and Analysis Process for Testability

Category	Checklist
Allocation of Responsibilities	Determine which system responsibilities are most critical and hence need to be most thoroughly tested. Ensure that additional system responsibilities have been allocated to do the following: ▪ Execute test suite and capture results (external test or self-test) ▪ Capture (log) the activity that resulted in a fault or that resulted in unexpected (perhaps emergent) behavior that was not necessarily a fault ▪ Control and observe relevant system state for testing Make sure the allocation of functionality provides high cohesion, low coupling, strong separation of concerns, and low structural complexity.
Coordination Model	Ensure the system's coordination and communication mechanisms: ▪ Support the execution of a test suite and capture the results within a system or between systems ▪ Support capturing activity that resulted in a fault within a system or between systems ▪ Support injection and monitoring of state into the communication channels for use in testing, within a system or between systems ▪ Do not introduce needless nondeterminism
Data Model	Determine the major data abstractions that must be tested to ensure the correct operation of the system. ▪ Ensure that it is possible to capture the values of instances of these data abstractions ▪ Ensure that the values of instances of these data abstractions can be set when state is injected into the system, so that system state leading to a fault may be re-created ▪ Ensure that the creation, initialization, persistence, manipulation, translation, and destruction of instances of these data abstractions can be exercised and captured
Mapping among Architectural Elements	Determine how to test the possible mappings of architectural elements (especially mappings of processes to processors, threads to processes, and modules to components) so that the desired test response is achieved and potential race conditions identified. In addition, determine whether it is possible to test for illegal mappings of architectural elements.
Resource Management	Ensure there are sufficient resources available to execute a test suite and capture the results. Ensure that your test environment is representative of (or better yet, identical to) the environment in which the system will run. Ensure that the system provides the means to do the following:

	▪ Test resource limits ▪ Capture detailed resource usage for analysis in the event of a failure ▪ Inject new resource limits into the system for the purposes of testing ▪ Provide virtualized resources for testing
Binding Time	Ensure that components that are bound later than compile time can be tested in the late-bound context. Ensure that late bindings can be captured in the event of a failure, so that you can re-create the system's state leading to the failure. Ensure that the full range of binding possibilities can be tested.
Choice of Technology	Determine what technologies are available to help achieve the testability scenarios that apply to your architecture. Are technologies available to help with regression testing, fault injection, recording and playback, and so on? Determine how testable the technologies are that you have chosen (or are considering choosing in the future) and ensure that your chosen technologies support the level of testing appropriate for your system. For example, if your chosen technologies do not make it possible to inject state, it may be difficult to re-create fault scenarios.

Now That Your Architecture Is Set to Help You Test. . .

By Nick Rozanski, coauthor (with Eoin Woods) of Software Systems Architecture: Working With Stakeholders Using Viewpoints and Perspectives

In addition to architecting your system to make it amenable to testing, you will need to overcome two more specific and daunting challenges when testing very large or complex systems, namely test data and test automation.

Test Data

Your first challenge is how to create large, consistent and useful *test data sets*. This is a significant problem in my experience, particularly for integration testing (that is, testing a number of components to confirm that they work together correctly) and performance testing (confirming that the system meets its requirements for throughput, latency, and response time). For unit tests, and usually for user acceptance tests, the test data is typically created by hand.

For example, you might need 50 products, 100 customers, and 500 orders in your test database, so that you can test the functional steps involved in creating, amending, or deleting orders. This data has to be sufficiently varied to make testing worthwhile, it has to conform to all the referential integrity rules and other constraints of your data model, and you need to be able to calculate and specify the expected results of the tests.

I've seen—and been involved in—two ways of doing this: you either write a system to generate your test data, or you capture a representative data set from the production environment and anonymize it as necessary. (Anonymizing test data involves removing any sensitive information, such as personal data about people or organizations, financial

details, and so on.)

Creating your own test data is the ideal, because you know what data you are using and can ensure that it covers all of your edge cases, but it is a lot of effort. Capturing data from the live environment is easier, assuming that there is a system there already, but you don't know what data and hence what coverage you're going to get, and you may have to take extra care to conform to privacy and data protection legislation.

This can have an impact on the system's architecture in a number of ways, and should be given due consideration early on by the architect. For example, the system may need to be able to capture live transactions, or take "snapshots" of live data, which can be used to generate test data. In addition, the test-data-generation system may need an architecture of its own.

Test Automation

Your second challenge is around *test automation*. In practice it is not possible to test large systems by hand because of the number of tests, their complexity, and the amount of checking of results that's required. In the ideal world, you create a test automation framework to do this automatically, which you feed with test data, and set running every night, or even run every time you check in something (the continuous integration model).

This is an area that is given too little attention on many large software development projects. It is often not budgeted for in the project plan, with an unwritten assumption that the effort needed to build it can be somehow "absorbed" into the development costs. A test automation framework can be a significantly complex thing in its own right (which raises the question of how you test it!). It should be scoped and planned like any other project deliverable.

Due consideration should be given to how the framework will invoke functions on the system under test, particularly for testing user interfaces, which is almost without exception a nightmare. (The execution of a UI test is highly dependent on the layout of the windows, the ordering of fields, and so on, which usually changes a lot in heavily user-focused systems. It is sometimes possible to execute window controls programmatically, but in the worst case you may have to record and replay keystrokes or mouse movements.)

There are lots of tools to help with this nowadays, such as Quick Test Pro, TestComplete, or Selenium for testing, and CruiseControl, Hudson, and TeamCity for continuous integration. A comprehensive list on the web can be found here: en.wikipedia.org/wiki/Test_automation.

10.4. Summary

Ensuring that a system is easily testable has payoffs both in terms of the cost of testing and the reliability of the system. A vehicle often used to execute the tests is the test harness. Test harnesses are software systems that encapsulate test resources such as test cases and test infrastructure so that it is easy to reapply tests across iterations and it is easy to apply the test infrastructure to new increments of the system. Another vehicle is the creation of test cases prior to the development of a component, so that developers know which tests their component must pass.

Controlling and observing the system state is a major class of testability tactics. Providing the ability to do fault injection, to record system state at key portions of the system, to isolate the system from its environment, and to abstract various resources are all different tactics to support the control and observation of a system and its components.

Complex systems are difficult to test because of the large state space in which their computations take place, and because of the larger number of interconnections among the elements of the system. Consequently, keeping the system simple is another class of tactics that supports testability.

10.5. For Further Reading

An excellent general introduction to software testing is [Beizer 90]. For a more modern take on testing, and from the software developer's perspective rather than the tester's, Freeman and Pryce cover test-driven development in the object-oriented realm [Freeman 09].

Bertolino and Strigini [Bertolino 96] are the developers of the model of testing shown in Figure 10.1.

Yin and Bieman [Yin 94] have written about executable assertions. Hartman [Hartman 10] describes a technique for using executable assertions as a means for detecting race conditions.

Bruntink and van Deursen [Bruntink 06] write about the impact of structure on testing.

Jeff Voas's foundational work on testability and the relationship between testability and reliability is worthwhile. There are several papers to choose from, but [Voas 95] is a good start that will point you to others.

10.6. Discussion Questions

1. A testable system is one that gives up its faults easily. That is, if a system contains a fault, then it doesn't take long or much effort to make that fault show up. On the other hand, fault tolerance is all about designing systems that jealously hide their faults; there, the whole idea is to make it very difficult for a system to reveal its faults. Is it possible to design a system that is both highly testable *and* highly fault tolerant, or are these two design goals inherently incompatible? Discuss.
2. "Once my system is in routine use by end users, it should *not* be highly testable, because if it still contains faults—and all systems probably do—then I don't want them to be easily revealed." Discuss.
3. Many of the tactics for testability are also useful for achieving modifiability. Why do you think that is?
4. Write some concrete testability scenarios for an automatic teller machine. How would you modify your design for the automatic teller machine to accommodate these scenarios?
5. What other quality attributes do you think testability is most in conflict with? What other quality attributes do you think testability is most compatible with?
6. One of our tactics is to limit nondeterminism. One method is to use locking to enforce synchronization. What impact does the use of locks have on other quality attributes?
7. Suppose you're building the next great social networking system. You anticipate that within a month of your debut, you will have half a million users. You can't pay half a million people to test your system, and yet it has to be robust and easy to use when all half a million are banging away at it. What should you do? What tactics will help you? Write a testability scenario for this social networking system.
8. Suppose you use executable assertions to improve testability. Make a case for, and then a case against, allowing the assertions to run in the production system as opposed to removing them after testing.